De la maquette à une représentation hiérarchique

Étude comparative de quatre approches de génération de code frontend

KOUONGA TADIE KENNEDY

Université de Douala

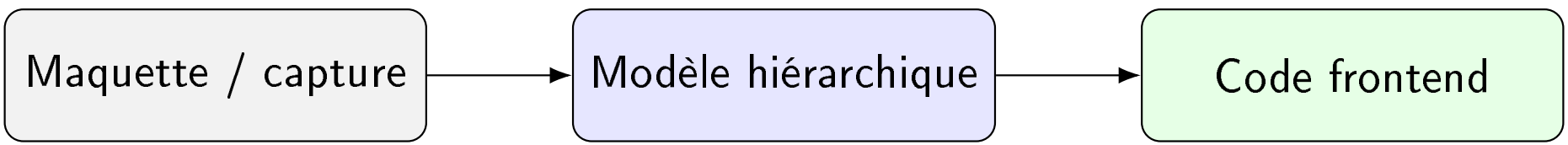
Plan

1. Question de recherche et périmètre de l'étude
2. Les quatre approches étudiées
3. Ce qui différencie leurs entrées
4. Comment mesurer la qualité d'une reconstruction
5. Comparaison chiffrée des quatre approches
6. Un mécanisme de correction automatique
7. Aspects pratiques : coût et bibliothèques de style
8. Limites actuelles et pistes de recherche

Question de recherche

Problème étudié

Comment passer d'une représentation visuelle d'une interface (maquette, capture d'écran, croquis) à une **représentation structurée et hiérarchique** qui pourra ensuite servir à générer automatiquement du code frontend ?



Périmètre exact de cette étude

Où se situe le travail présenté

Cette étude se concentre uniquement sur la **première flèche** du schéma précédent : comment transformer la maquette en modèle hiérarchique (arbre de composants, arbre de mise en page).



La génération finale du code (deuxième flèche) n'est pas l'objet direct de la comparaison : elle sert seulement à évaluer, *a posteriori*, si le modèle hiérarchique construit était de bonne qualité.

Pourquoi une représentation intermédiaire ?

- Une image décrit surtout **ce que l'on voit**.
- Le code doit exprimer **comment les éléments sont organisés**.
- Il faut donc retrouver les éléments, leurs relations, leur organisation spatiale et leurs comportements de mise en page.

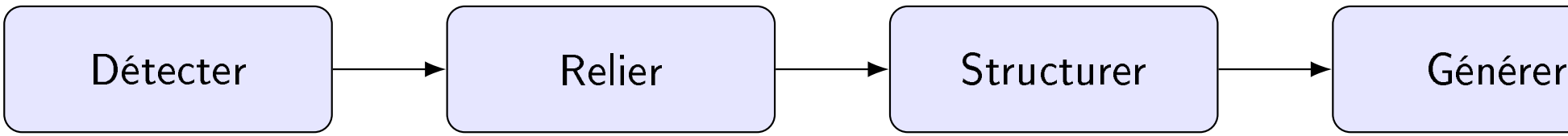
Point important

« Représentation hiérarchique » est utilisée au sens large : arbre de composants, arbre de mise en page, arbre de subdivisions ou structure avec contraintes.

Les quatre approches étudiées

Approche	**Idée principale**
Prototype2Code	Reconstruire une hiérarchie à partir des relations de contenance, puis organiser en lignes/colonnes
DCGen	Diviser récursivement l'image en segments ; la division forme un arbre
ScreenCoder	Détecter et nommer les grandes régions, puis construire un arbre selon des règles de conception web
DesignCoder	Diviser l'interface, extraire le sens des éléments, puis les regrouper dans un arbre de composants

Ce que les quatre méthodes ont en commun



1. Identifier des unités de l'interface (éléments, régions, widgets)
2. Déterminer des relations entre ces unités
3. Construire une structure (ne plus traiter l'image comme plate)
4. Utiliser cette structure pour générer le code

Deux familles d'entrées

Avant de comparer les résultats, il faut préciser ce que chaque méthode reçoit réellement en entrée : cela change la difficulté du problème résolu.

Avec métadonnées de conception

- Prototype2Code
- DesignCoder

Image + fichier Figma/Sketch (id, type, position, style déjà connus)

Image seule

- DCGen
- ScreenCoder

Aucune donnée cachée : tout doit être deviné depuis les pixels

Comme monter un meuble avec la notice de montage, contre le monter seulement à partir d'une photo du résultat fini.

Comment mesurer si une reconstruction est bonne ?

Deux questions distinctes se posent, et chacune a sa propre façon d'être mesurée :

1. Est-ce que ça *ressemble* à l'original ?

On compare les images (originale contre générée) — similarité **visuelle**.

2. Est-ce que c'est *bien rangé* à l'intérieur ?

On compare les arbres de composants (référence contre généré) — similarité **structurelle**.

Deux sites peuvent se ressembler visuellement tout en ayant un code totalement désorganisé derrière — d'où l'intérêt de mesurer les deux séparément.

Les métriques de similarité visuelle, expliquées simplement

CLIP score

Un réseau de neurones compare l'image originale et l'image générée et donne un score entre 0 et 1 : plus le score est proche de 1, plus les deux images « veulent dire la même chose » visuellement.

Block-match / Texte / Position / Couleur

On découpe les deux images en blocs (un bouton, un titre, une carte...), on les fait correspondre un à un, puis on vérifie si chaque bloc a la bonne taille, le bon texte, la bonne position et la bonne couleur.

Les métriques de similarité structurelle, expliquées simplement

TreeBLEU

Compare l'arbre du code généré à l'arbre de référence, morceau par morceau (un peu comme on compterait combien de groupes de mots se retrouvent à l'identique entre deux phrases).

Container Match

Vérifie si les éléments ont été regroupés dans les bons « contenants » (le bon parent), sans juger les détails à l'intérieur — juste : est-ce que ce bouton a bien été rangé dans la bonne carte ?

Tree Edit Distance (TED)

Nombre minimum d'opérations (ajouter, supprimer, renommer un nœud) nécessaires pour transformer l'arbre généré en l'arbre de référence. **Plus ce nombre est bas, mieux c'est.**

Fidélité visuelle : les scores CLIP

Méthode	**Score CLIP (ressemblance visuelle)**
ScreenCoder	81 %
DesignCoder	92 %
DCGen	86 %
Prototype2Code	non publié dans le document source

Ces scores proviennent des évaluations propres à chaque article et permettent une première comparaison quantitative, à interpréter en tenant compte de la différence d'entrées vue plus haut.

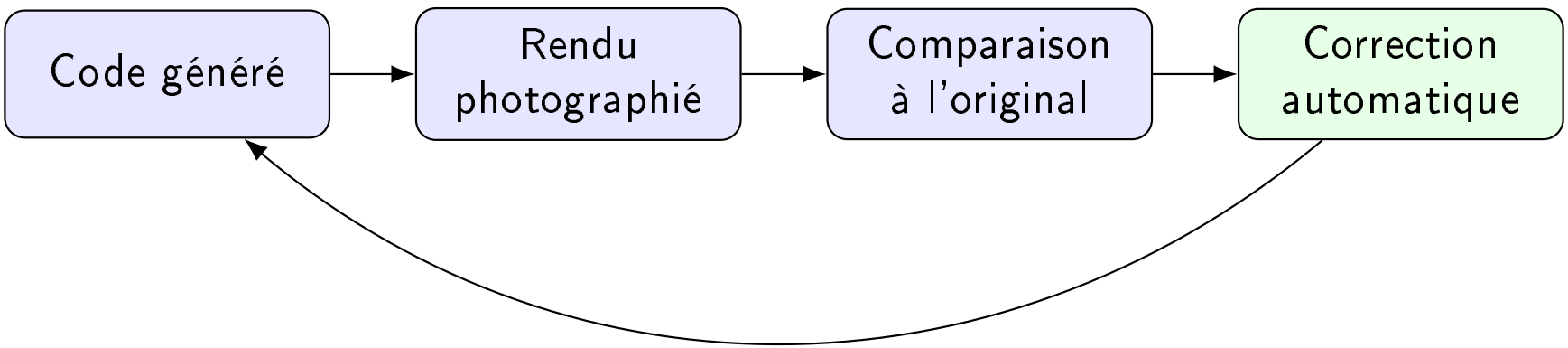
Fidélité structurelle : au-delà de l'apparence

Méthode	TreeBLEU	Container Match	TED
Prototype2Code	0.53	0.58	40.0
DesignCoder	**0.69**	**0.75**	**28.2**

*DesignCoder obtient un score supérieur sur les trois métriques structurelles : son arbre de composants est non seulement visuellement fidèle, mais aussi mieux organisé — notamment parce qu'il cherche à comprendre le **rôle** de chaque élément avant de le ranger.*

Un mécanisme de correction automatique

Une des quatre méthodes ne se contente pas de produire le code en une seule passe : elle vérifie et corrige son propre résultat.



Sans ce module, les scores de DesignCoder chutent nettement (CLIP : 0.92 → 0.88 ; TreeBLEU : 0.66 → 0.55) — ce qui confirme que cette boucle de vérification a un effet mesurable sur la qualité finale.

Coût et bibliothèques de style

Bibliothèques de composants

ScreenCoder génère directement en **Tailwind CSS**, une bibliothèque de styles standard, plutôt que du CSS entièrement fait main.

Temps de génération

ScreenCoder discute explicitement le compromis **coût contre qualité** : un modèle plus petit permet un rendu rapide et approximatif, un modèle plus grand prend plus de temps (de l'ordre de quelques dizaines de secondes) pour un résultat plus fidèle.

Ce qui reste ouvert

Aucune des quatre méthodes ne gère de design system personnalisé (tokens de couleur ou de typographie propres à une marque).

Limite : une image figée dans le temps

Les quatre méthodes construisent un arbre à partir d'une image **statique**, à un instant donné.

- Aucune ne modélise le survol de la souris, l'ouverture d'un menu ou d'une fenêtre modale.
- Aucune ne modélise l'adaptation dynamique à différentes tailles d'écran, au-delà de la mise en page initiale.

Limite : l'accessibilité n'est pas prise en compte

Aucune des quatre méthodes ne mentionne les normes d'accessibilité :

- Pas de rôles sémantiques HTML5 profonds (`main`, `nav`, `article`).
- Pas de prise en compte des lecteurs d'écran ni de la navigation au clavier.

Ce constat vaut pour l'ensemble des quatre approches, pas seulement pour certaines d'entre elles.

Limite : dépendance à un langage cible unique

Méthode	**Technologie cible unique**
DesignCoder	React Native uniquement
Prototype2Code, DCGen, ScreenCoder	HTML/CSS uniquement

Aucune méthode ne produit un format intermédiaire réellement indépendant de la technologie de sortie.

Conclusion et pistes de recherche

Synthèse

Les travaux étudiés convergent vers une même idée : la maquette doit être transformée en une représentation structurée avant de produire le code. La qualité de cette représentation intermédiaire conditionne directement la qualité du code final.

Question qui se dégage

Quel niveau d'information doit contenir notre modèle pour être indépendant de la technologie cible tout en restant suffisamment riche pour générer ensuite React, HTML/CSS, Flutter, etc. ?

Trois pistes concrètes

1. Un format intermédiaire réellement indépendant du langage cible
2. Une gestion des états d'interface (survol, clic, responsive)
3. Une prise en compte de l'accessibilité dès la construction de l'arbre

Références principales

- S. Xiao et al. *Prototype2Code*. arXiv :2405.04975, 2024.
- Y. Wan et al. *Divide-and-Conquer : Generating UI Code from Screenshots*. Proc. ACM Softw. Eng., FSE, 2025.
- Y. Jiang et al. *ScreenCoder*. arXiv :2507.22827, 2025.
- Y. Chen et al. *DesignCoder : Hierarchy-Aware and Self-Correcting UI Code Generation*. arXiv :2506.13663, 2025.